

Práctica Aprendizaje Automático I

Yixuan Lu Guo
Jennifer Juez Yanguas

Noviembre de 2025

1. Introducción

En la siguiente práctica abordamos un problema de clasificación de salarios a partir de un conjunto de datos multivariante, siguiendo la metodología estándar en el Aprendizaje Automático supervisado.

Primero, nos centramos en el proceso de limpieza y preprocesamiento de los datos, tal y como detallamos en la Sección 2. En esta fase, gestionamos valores nulos, aplicamos *One-Hot Encoding* (OHE) a las variables categóricas, y abordamos el desequilibrio de clases utilizando la técnica de sobremuestreo **SMOTE** en el conjunto de entrenamiento. También, gestionamos los valores atípicos del dataset. Además, realizamos una selección de características rigurosa utilizando el método **RFECV** (Recursive Feature Elimination with Cross-Validation) con Regresión Logística, con el objetivo de optimizar el rendimiento y la estabilidad de los modelos. Por último, guardamos datasets escalados con `StandardScaler`, `MinMaxScaler` y sin escalar, para poder utilizarlos en el notebook "supervised.ipynb", de modo, que dependiendo del modelo que vayamos a entrenar, utilice un dataset u otro.

En la segunda etapa, nos dedicamos a la predicción supervisada. Probamos con diversos modelos de clasificación vistos en clase (árboles de decisión, Regresión Logística, Support Vector Machines, etc.), ajustando y optimizando sus respectivos hiperparámetros mediante técnicas de búsqueda adecuadas (*grid search*). Dada la complejidad computacional, hemos utilizado una validación cruzada reducida ($CV=3$). La elección del modelo final se fundamentará en la métrica de evaluación utilizada en la competición: **F1-score**. Esta métrica resulta ideal para problemas de clasificación con desbalance de clases, ya que equilibra la precisión (*Precision*) y la exhaustividad (*Recall*). Finalmente, generaremos las predicciones con el modelo óptimo para su presentación.

2. Justificaciones del Preprocesamiento de Datos

2.1. 1. Limpieza y Preparación Inicial de Datos

2.1.1. Decisión 1.1: Renombrado de Columnas

- Renombramos las columnas con nombres ambiguos como `trabajo.1` y `trabajo` a nombres más descriptivos (`sector` y `tipo_trabajo`).
- Esto mejora la comprensión de las características del *dataset*.

2.1.2. Decisión 1.2: Manejo de Valores Faltantes ('?')

- Reemplazamos los valores faltantes, representados por `¿`, por `np.nan`.
- Imputamos en los valores nulos la moda de cada columna categórica (`tipo_trabajo`, `sector`, `pais-origen` son las columnas con valores faltantes).
- Elegimos la moda por ser apropiada para rellenar huecos en variables categóricas, ya que introduce el valor más frecuente, manteniendo la distribución original de la columna.

2.1.3. Decisión 1.3: Codificación de la Variable Objetivo (salario)

- Codificamos la variable objetivo `salario` (con etiquetas `<=50K` y `>50K`) a valores numéricos 0 y 1 utilizando `LabelEncoder`.
- Esto es necesario para que los modelos de *machine learning* puedan procesar la variable a predecir.

2.1.4. Decisión 1.4: Manejo de Datos Duplicados/Inconsistentes

- Decidimos conservar los duplicados porque el *dataset* contiene variables demográficas (edad, educación, país, etc.). Es muy probable que existan múltiples personas (filas) con combinaciones idénticas de atributos que son válidas y representan la realidad de la población. Además, eliminar duplicados en datos podría sesgar la distribución natural del *dataset* y afectar negativamente la capacidad de generalización del modelo.

2.1.5. Decisión 1.5: Tratamiento de Valores Atípicos (*Outliers*) mediante Winsorización

- Los diagramas de caja nos muestran a continuación la presencia de valores atípicos severos en las columnas numéricas. En lugar de eliminar filas (lo que sesgaría los datos, ya que los extremos son cruciales para la clase minoritaria >50K), aplicamos **Winsorización**.
- Esta técnica limita los valores de las columnas numéricas (típicamente -y en nuestro caso- en los percentiles 5 y 95) ajustándolos al valor de ese percentil. Esto mitiga la influencia extrema de los *outliers* en modelos sensibles al gradiente sin perder información.

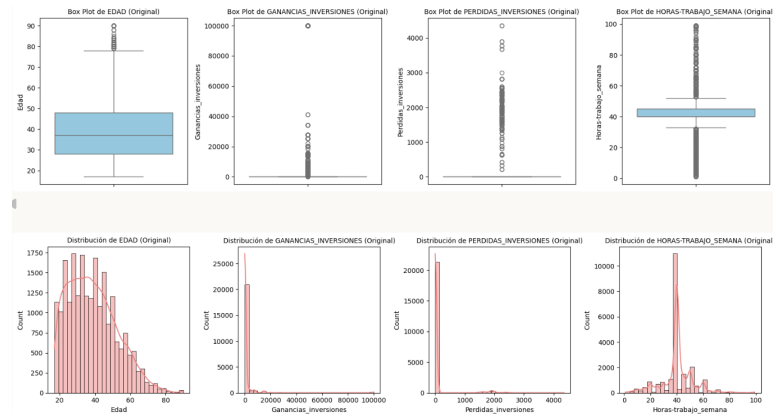


Figura 1: Box Plots e Histogramas (asimetría por outliers).

2.1.6. Decisión 1.6: Tratamiento de Varianza Nula en Columnas de Inversión

- Durante el análisis de correlación posterior al preprocesamiento, nos dimos cuenta de que la columna **num_perdidas-inversiones** nos mostraba un valor de correlación NaN con el resto de *features*.
- Al verificar la varianza, pudimos confirmar que **su valor era 0.0**. Este resultado se debe a que la columna presenta una **dispersión extrema** en el conjunto de entrenamiento: más del **95 %** de sus registros son **cero** (+25000). Aunque **num_ganancias-inversiones** no llega a varianza cero, tiene una dispersión similar (+25000 valores son 0).

- Una variable con varianza nula es una **constante** y no aporta ninguna capacidad discriminatoria ni predictiva a los modelos.
- Este hallazgo **refuerza la necesidad** de aplicar la **Winsorización** a ambas columnas de capital. La Winsorización mitiga el impacto de los *outliers* no-cero (los pocos valores que sí varían y son relevantes) sin perder esas instancias vitales para la clase minoritaria, manteniendo el *dataset* estable para los modelos lineales.

2.2. 2. Estrategia de Preprocesamiento y *Data Leakage*

2.2.1. Decisión 2.1: *Split* (División) Antes de la Transformación

- Realizamos la división de los datos en conjuntos de entrenamiento (**Train**) y validación interna (**Test/Valid**) antes de aplicar cualquier transformación.
- Hacemos la división de forma estratificada (**stratify=y_encoded**) para asegurar que la proporción de las clases de salario se mantenga igual en ambos conjuntos.
- El motivo principal es evitar la fuga de datos (*data leakage*). Si un transformador (como **OneHotEncoder** o un **Scaler**) aprende del conjunto de validación, estamos usando información del futuro para preparar el entrenamiento, lo que resultaría en una evaluación sobre-optimista.
- La regla que seguimos es que el **fit()** de los transformadores solo se hace con los datos de entrenamiento.

2.2.2. Decisión 2.2: Uso de *One-Hot Encoding* (OHE) y **ColumnTransformer**

- Utilizamos **OneHotEncoder** para transformar las variables categóricas en representaciones binarias (*dummies*).
- Esto es fundamental porque muchos algoritmos no pueden trabajar directamente con valores categóricos no ordenados.
- Usamos **ColumnTransformer** para aplicar OHE solo a las columnas categóricas, dejando las numéricas sin escalar (**passthrough**).

2.2.3. Decisión 2.3: Gestión de Valores Categóricos Desconocidos

- Dentro del `ColumnTransformer` y al aplicar `OneHotEncoder` a las variables categóricas, configuramos el parámetro `handle_unknown='ignore'`.
- Esta configuración nos aporta robustez al modelo, ya que se asegura de que si una categoría aparece por primera vez en test (Kaggle) y no aparecía en el dataset de entrenamiento, el encoder la ignore sin provocar un error, asignándole un vector de ceros.
- De este modo, el modelo puede procesar el conjunto de test de Kaggle sin romperse, manteniendo la integridad del proceso de predicción.

2.3. 3. Manejo del Desbalanceo de Clases

2.3.1. Decisión 3.1: Elección de *Oversampling* (Sobremuestreo)

- El desbalanceo es significativo, con la clase minoritaria (>50K) representando solo un $\sim 24\%$ del total.
- Elegimos una técnica de sobremuestreo (*oversampling*) como **SMOTE**.
- Descartamos el *undersampling* porque implicaría descartar una gran cantidad de datos de la clase mayoritaria ($\sim 76\%$), lo que llevaría a una pérdida de información y afectaría a la capacidad de generalización del modelo.
- Preferimos SMOTE (Synthetic Minority Over-sampling Technique) porque crea nuevos ejemplos sintéticos cerca de la frontera de decisión, ayudando al modelo a generalizar mejor y reduciendo el riesgo de sobreajuste.

2.3.2. Decisión 3.2: Aplicación de SMOTE Solo al *Train*

- Aplicamos la técnica de sobremuestreo (SMOTE) solo al conjunto de entrenamiento (`X_train_processed_df`, `y_train`).
- Evaluamos el modelo en un conjunto que refleje la distribución real de las clases en la población, para obtener métricas realistas.

2.4. 4. Selección de Características

2.4.1. Decisión 4.1: Elección del Método RFECV con Regresión Logística

- Tras evaluar varios métodos, el método RFECV (Recursive Feature Elimination with Cross-Validation) (Wrapper) utilizando un estimador de Regresión Logística y optimizado para el *F1-score*, ha conseguido la mejor puntuación.
- Lo bueno de este método es que evalúa interacciones (entrenamos el modelo de Regresión Logística y evaluamos las características basándonos en su rendimiento real, teniendo en cuenta las interacciones y la colinealidad); optimiza la métrica objetivo (optimizamos directamente F1-score, asegurando que las características seleccionadas son las que mejor contribuyen a ese rendimiento) y determina la *K* óptima (utilizamos *Cross-Validation* para encontrar el número óptimo de características a conservar).

2.4.2. Decisión 4.2: Estandarización de Numéricas para RFECV

- Aplicamos `StandardScaler` (Estandarización) solo a las columnas numéricas (`num_*`) antes de ejecutar RFECV con Regresión Logística porque la Regresión Logística se basa en la magnitud de los coeficientes. El escalado asegura que las variables no dominen el proceso de selección o entrenamiento por tener rangos de valores mayores.

2.4.3. Decisión 4.3: Análisis de Correlación (Heatmap)

- Hemos incluido un *heatmap* de la correlación de Pearson entre las *features* numéricas y el *Target*. La gráfica nos muestra que la variable `num_ganancias-inversiones` (0.31) es el predictor numérico más fuerte (un valor moderado).
- Al confirmar la baja correlación entre las *features* numéricas, el uso de la **Regularización L1** en el modelo ganador se justifica para manejar la alta dimensionalidad y la posible colinealidad generada por las 80+ variables OHE y las interacciones polinomiales.

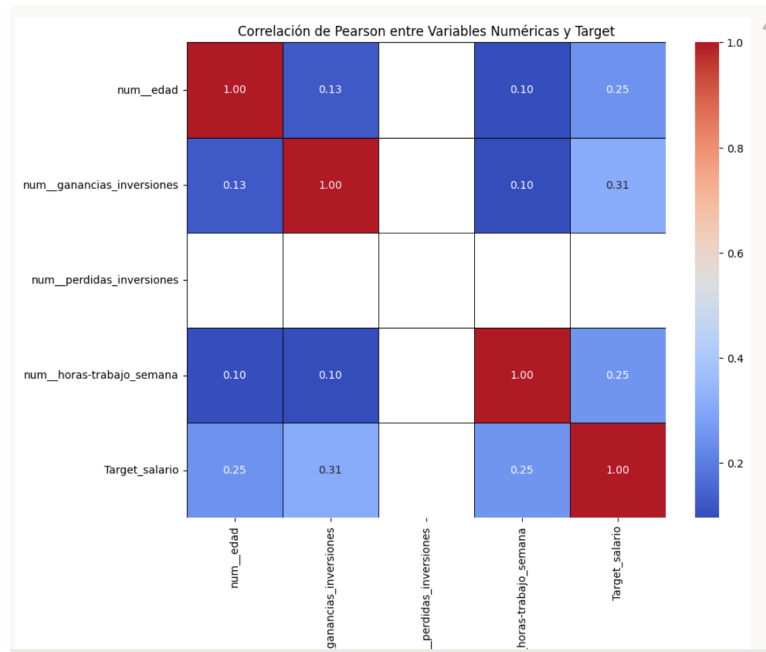


Figura 2: Heatmap de correlación entre features.

2.5. 5. Estructura de Salida y Metadatos

2.5.1. Decisión 5.1: Estructura de Salida con Escalado Preaplicado

- No incluimos el escalado final en los archivos CSV base (`_ohe.csv` y `_rfecv_lr.csv`), manteniendo estos conjuntos con One-Hot Encoding (OHE) y *features* numéricas sin tocar.
- Guardamos versiones escaladas separadas (`_std.csv` y `_minmax.csv`) ya proyectadas a las *features* seleccionadas por RFECV. Esto permite al *notebook* de modelado (`supervised.ipynb`) cargar directamente el conjunto de datos apropiado para cada modelo (ej., el csv con Min-Max para kNN o el csv con Standard para SVM/Regresión Logística), simplificando la definición del *pipeline* posterior. El escalado lo hemos aplicado **solo** usando el fit en el conjunto de entrenamiento original para evitar la fuga de datos (*data leakage*).

2.5.2. Decisión 5.2: Creación de Múltiples Archivos de Salida (ZIP)

- Generamos dos versiones de los datos (OHE completo y OHE proyectado por RFECV-LR), tanto para *train*, *valid* y *kaggle*.
- Esto permite a futuros experimentos probar otros métodos de selección de características o utilizar directamente la mejor selección de *features* encontrada.

3. Justificaciones del Modelado Supervisado

3.1. 6. Configuración de *Pipelines* y Estrategia de *Grid Search*

3.1.1. Decisión 6.1: Uso de ImbPipeline

- Construimos un ImbPipeline (Pipeline de imblearn) para cada modelo en lugar del Pipeline estándar de scikit-learn.
- Esto nos permite encapsular correctamente el preprocesamiento, el balanceo de clases y el clasificador final en una única secuencia de pasos.
- El uso de ImbPipeline garantiza que la técnica de sobremuestreo (SMOTE) se aplique (solo al conjunto de entrenamiento) dentro de cada *fold* (pliegue) de la validación cruzada, evitando la fuga de datos durante el ajuste del modelo.
- A continuación se presenta la justificación resumida de la selección de hiperparámetros para la configuración de los modelos.

La estrategia de búsqueda se centra en tres ejes principales:

- **Manejo del Desbalance:** Explorar la interacción entre el sobremuestreo (SMOTE) y el ajuste de pesos (`class_weight`).
- **Preprocesamiento y Escalado:** Aplicar el escalado adecuado según la sensibilidad de cada algoritmo.
- **Regularización y Complejidad:** Ajustar los parámetros clave de cada modelo para prevenir el *overfitting* y optimizar su rendimiento.

Estrategia Común de Manejo del Desbalance:

Para casi todos los modelos, se incluyeron dos parámetros en el *grid search* para abordar sistemáticamente el desbalance de clases:

- **smote**: Se prueba la inclusión de SMOTE (con **k_neighbors=3**) frente a "passthrough" (no aplicar sobremuestreo). Esto permite al *grid search* determinar si la generación de datos sintéticos mejora el modelo o si es preferible usar los datos originales.
- **clf__class_weight**: Se prueban diferentes configuraciones [**None**, {0: 1.25, 1: 1.0}, ...]. Esto evalúa si el *cost-sensitive learning* (asignar un coste mayor a la clase mayoritaria, compensando el desbalance o el propio SMOTE) es beneficioso, frente a tratar todas las clases por igual (**None**).

Justificación por Tipo de Modelo y Escalado:

Los modelos se agruparon según su necesidad de escalado de datos, siguiendo las mejores prácticas de cada algoritmo.

A. Modelos con Standard Scaler (Media 0, Varianza 1)

Estos modelos (SVM Lineal, Logística, SVM RBF) son sensibles a la escala de las *features* porque sus algoritmos se basan en cálculos de gradiente o distancias (kernels). **StandardScaler** es el preprocesamiento estándar para ellos.

SVM Lineal (SGD) y Regresión Logística:

- **clf__alpha / clf__C**: Parámetros de *regularización*. Se prueban diferentes intensidades (valores bajos, medios, altos) para encontrar el equilibrio óptimo entre ajuste y generalización.
- **clf__penalty**: Se prueba 11 (Lasso, en Logística) para ver si la selección de *features* dispersa (*sparse*) es útil, y 12 (Ridge) para una regularización general.

SVM RBF (Kernel no lineal):

- **clf__C**: Parámetro de regularización que controla el margen del SVM.
- **clf__gamma**: Se usa "scale", que es un valor robusto y estándar que se ajusta automáticamente según la varianza de los datos.

Logística Polinómica:

- **prep__num__poly__degree**: Se fija en [2] para probar explícitamente si la inclusión de interacciones y términos cuadráticos (solo en variables numéricas) aporta valor predictivo al modelo logístico.

B. Modelos con MinMax Scaler (Rango [0, 1])

Estos modelos (kNN y Redes Neuronales) funcionan mejor cuando todas las *features* están estrictamente en el mismo rango.

k-Nearest Neighbors (kNN):

- **Justificación de Escalado:** `MinMaxScaler` es crucial para kNN, ya que el algoritmo se basa puramente en distancias. Si no se escala, las variables con magnitudes grandes dominarían el cálculo de la distancia.
- `clf__n_neighbors`: Se prueban valores [15, 21, 31] (impares, para evitar empates) para encontrar el tamaño de vecindario óptimo, balanceando entre ruido (k bajo) y exceso de suavizado (k alto).

Redes Neuronales (MLP sklearn y Keras):

- **Justificación de Escalado:** Las MLPs convergen mucho más rápido y de forma más estable cuando las entradas están normalizadas, especialmente en el rango [0, 1] para activaciones como ReLU.
- `clf__hidden_layer_sizes`: Se prueban diferentes *arquitecturas* (p.ej., (64, 64), (100,)) para testar la capacidad del modelo (más ancho vs. más profundo).
- `clf__alpha` / `clf__model__dropout_rate`: Son las técnicas de *regularización* específicas (L2 y Dropout) para prevenir el *overfitting* en las redes neuronales.

C. Modelos sin Escalado (Basados en Árboles)

Estos modelos (Decision Tree, Random Forest) son *invariantes a la escala* de las *features*, ya que toman decisiones basadas en umbrales ("feature X > 5.0") y no en la magnitud de las variables.

Decision Tree y Random Forest:

- `clf__max_depth`, `clf__min_samples_leaf`, `clf__min_samples_split`: Son los parámetros clave de *poda* (*pruning*). Se ajustan para controlar la complejidad de los árboles y evitar que memoricen los datos de entrenamiento (*overfitting*).
- `clf__n_estimators` (Solo Random Forest): Se prueba el número de árboles en el ensamblado ([100, 200]) para balancear la robustez del modelo con el coste computacional.

Decisión 6.2: Adaptación del Escalado al Modelo

- Mantenemos el escalado como parte de cada *pipeline* y lo ajustamos a la naturaleza del clasificador.
- Para modelos basados en la distancia o magnitud de características (Regresión Logística, SVM, *Linear SVM*, MLP), utilizamos `StandardScaler` o `MinMaxScaler` (por ejemplo, `MinMaxScaler` para kNN y MLP).
- Para el Árbol de Decisión, utilizamos `make_prep("none")` (`passthrough`), ya que los modelos basados en árboles no son sensibles al escalado de las variables numéricas.

Decisión 6.3: Entrenamiento y Búsqueda de Hiperparámetros (*Grid Search*)

- Entrenamos cada *pipeline* y buscamos el conjunto óptimo de hiperparámetros (*best_params*) utilizando `GridSearchCV` con validación cruzada. Como dijimos anteriormente, utilizamos `CV=3` en lugar del estándar `CV=5` por temas de alto coste computacional (introducido por el sobremuestreo SMOTE y los modelos complejos (SVM, MLP)).
- Utilizamos un enfoque de **evaluación multi-métrica**, calculando *F1-score*, *Accuracy*, *Precision*, *Recall*, *ROC-AUC* y *PR-AUC* en la validación cruzada.
- **Estrategia de Ajuste (Refit) por Precisión:** Aunque el objetivo final es el **F1-score**, ajustamos (`refit`) el `GridSearchCV` utilizando la métrica **Precisión** (`refit=Precision`). Esto fuerza a cada modelo a encontrar la frontera de decisión más "conservadora" que minimice los **Falsos Positivos (FP)**, optimizando así el *trade-off* para un mejor F1-score. Este modo nos daba los mismos resultados que utilizando F1 en `refit`, por lo que decidimos dejar `REFIT = "f1"`.
- Selección Final por F1-score: Posteriormente, la elección del modelo final se basa en la métrica F1-score obtenida en el conjunto de validación. Esto garantiza que el modelo elegido tenga un buen balance entre Precisión y Recall, cumpliendo con el objetivo de la competición de Kaggle.

3.2. 7. Análisis de Resultados y Decisión Final

3.2.1. Decisión 7.1: Modelos Complejos (Redes Neuronales)

- Redes Neuronales (MLP): Incluimos dos implementaciones: la versión nativa de `scikit-learn` (`mlp_minmax`) y una versión avanzada con `KerasClassifier` (`mlp_keras_minmax`), para la cual hemos utilizado las librerías de Keras y TensorFlow. La implementación de Keras aborda explícitamente el requisito de usar Dropout y capas Fully Connected, usando Dropout y regularización L2 (α) como herramientas para evitar el sobreajuste.

3.2.2. Decisión 7.2: Elección del Modelo Ganador (*Random Forest*)

- Ordenamos todos los modelos por su rendimiento en el conjunto de **validación interna** (`VALID_F1`) y seleccionamos el de mayor puntuación, que resultó ser el **Random Forest Unscaled** (F1: 0,7014).
- Este modelo, al ser un **ensemble** de árboles, ha demostrado ser más robusto y estable que el Árbol de Decisión individual (en séptima posición) y que la Regresión Polinomial.
- El Random Forest supera a la Regresión Polinomial gracias a su capacidad de generalización en datos tabulares complejos.
- **Análisis del Trade-off:** El F1-score más alto se obtiene porque el Random Forest encuentra el equilibrio óptimo entre Precisión y Recall (el menor número de Falsos Positivos y Falsos Negativos combinados) en la Matriz de Confusión final.

3.2.3. Decisión 7.3: Análisis de Métricas y Rendimiento

- La *Accuracy* de todos los modelos es alta (alrededor de 0,78 a 0,83), pero esta métrica es poco informativa debido al desbalanceo de clases.
- La métrica clave para la clasificación desbalanceada es el **F1-score** y el **PR-AUC**. El PR-AUC confirma la robustez del Random Forest, ya que lidera el ranking con un valor de 0,77 (el más alto para la clase minoritaria).

- El AUC-ROC es superior a 0.91 para el Random Forest y el Polinomial, confirmando la alta capacidad de discriminación.
- La brecha entre el CV_F1 y el $VALID_F1$ es muy baja, lo que indica que **no hay sobreajuste** evidente en el modelo ganador.
- kNN obtiene los peores resultados ($Valid\ F1 \approx 0,635$, $PR-AUC \approx 0,62$) por su sensibilidad a la alta dimensionalidad del dataset OHE (se basa en distancias).

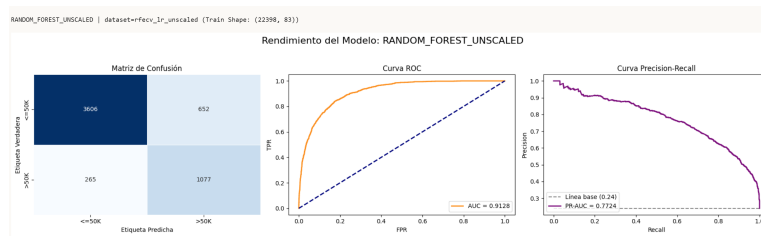


Figura 3: Gráficas Random Forest (mejor modelo).

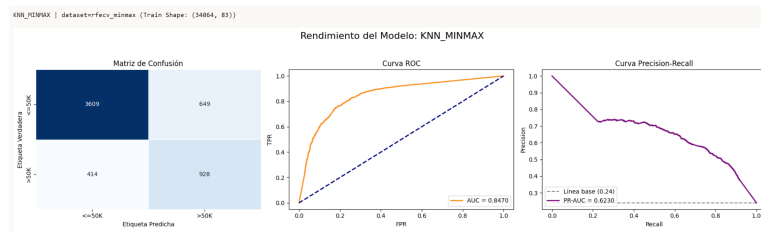


Figura 4: Gráficas KNN (peor modelo).

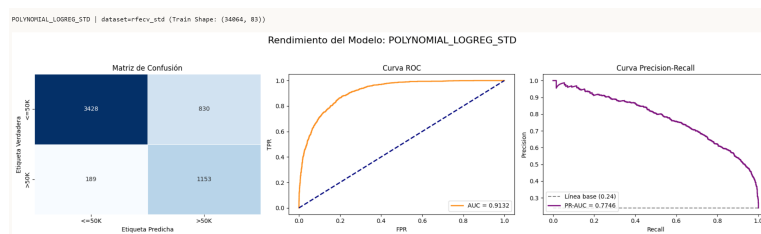


Figura 5: Gráficas Regresión Polinomial.

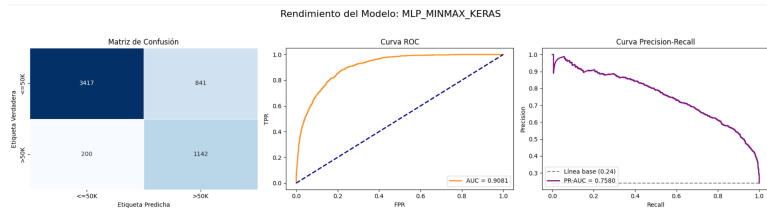


Figura 6: Gráficas Red Neuronas Keras.

	model	dataset	best_params	cv_precision	cv_f1	cv_recall	cv_accuracy	cv_rec_auc	cv_pr_auc	valid_precision	valid_f1	valid_recall	valid_accuracy	valid_mcc	valid_rec_auc	valid_pr_auc
0	random_forest_unscaled	rftcv_0_unscaled	{'rf_class_weight': (0, 1.25, 1, 10), 'rf_...'	0.620154	0.683121	0.760717	0.810878	0.900359	0.741754	0.622903	0.701400	0.802534	0.836250	0.600052	0.912797	0.772407
1	svm_rbf_std	rftcv_std	{'rf_C': 3, 'rf_class_weight': None, 'rf_...'	0.843419	0.875634	0.910483	0.870685	0.947140	0.946824	0.610891	0.693720	0.802534	0.830179	0.589526	0.897828	0.725819
2	polynomial_logreg_std	rftcv_std	{'rf_C': 1, 'rf_class_weight': None, 'rf_...'	0.819095	0.844240	0.871008	0.839102	0.916604	0.905605	0.581442	0.693534	0.859165	0.818036	0.592881	0.913222	0.774581
3	mlp_minmax	rftcv_minmax	{'rf_activation': 'relu', 'rf_alpha': 0.00...	0.849725	0.868370	0.888092	0.865430	0.942476	0.937586	0.625607	0.689632	0.768256	0.834286	0.583883	0.899571	0.730183
4	mlp_minmax_keras	rftcv_minmax	{'rf_class_weight': None, 'rf_model_alpha...'	0.819740	0.840007	0.861614	0.835926	0.915481	0.902504	0.575895	0.686917	0.850969	0.814107	0.583259	0.908059	0.758023
5	logistic_std	rftcv_std	{'rf_C': 1.0, 'rf_class_weight': None, 'rf_...'	0.807962	0.834907	0.863727	0.829204	0.908295	0.893295	0.560867	0.675171	0.847988	0.804664	0.567236	0.902585	0.745866
6	decision_tree_unscaled	rftcv_0_unscaled	{'rf_max_depth': ...}	0.581828	0.657721	0.758674	0.810876	0.878246	0.694474	0.598955	0.673409	0.768001	0.821250	0.561146	0.895416	0.711431
7	linear_svm_std	rftcv_std	{'rf_alpha': 0.001, 'rf_class_weight': None, 'rf_...'	0.798548	0.832398	0.881517	0.822311	0.903400	0.887964	0.533272	0.660040	0.865872	0.766250	0.548965	0.905272	0.735368
8	knn_minmax	rftcv_minmax	{'rf_n_neighbors': 15, 'rf_p': 1, 'rf_w...'	0.847926	0.852041	0.856212	0.851309	0.913093	0.886277	0.588409	0.635834	0.691505	0.810179	0.511619	0.847033	0.622968

Figura 7: Tabla comparativa de los resultados de los modelos probados.

4. Justificaciones del Modelo Ganador

4.1. 8. Implementación y Exportación Final

4.1.1. Decisión 8.1: Refitting y Predicción Final

- Nos quedamos con la **configuración óptima** del modelo **Random Forest** encontrada en la validación interna.
- Refiteamos el mejor estimador (`best_estimator_all`) con el conjunto completo de datos de **entrenamiento + validación** (`X_train_best + X_valid_best`), asegurando la coherencia de escalado al utilizar el dataset específico del modelo ganador.
- Implementamos la visualización de la Matriz de Confusión y la Importancia de Características del modelo ganador para justificar el resultado y analizar la distribución de errores.
- Finalmente, utilizamos este estimador reentrenado para realizar las predicciones sobre el conjunto de **test externo de Kaggle** (`X_kaggle`) y generamos el archivo `test_labels.csv` con las etiquetas de salario en formato de texto.

4.1.2. Decisión 8.2: Implementación de la Matriz de Confusión y Feature Importance

- La visualización de la Matriz de Confusión sobre el conjunto de validación (`X_valid`) confirma el punto de equilibrio del modelo: Muestra un alto número de Verdaderos Positivos (TP) y un bajo número de Falsos Negativos (FN) (personas que ganan $>50K$, pero el modelo predice $\leq 50K$). Este bajo FN es la clave del alto Recall ($\approx 0,85$), esencial para maximizar el F1-score en este problema desbalanceado.
- El modelo sacrifica ligeramente la Precisión, acepta cierto número de Falsos Positivos (FP), un trade-off necesario para la detección óptima de la clase minoritaria.
- El gráfico de importancia de características de Random Forest nos muestra que los factores categóricos son los predictores más importantes, ya que features como el estado civil (Casado/a) y la posición familiar (Marido) dominan el Top 10. El modelo se apoya principalmente en estos indicadores sociodemográficos altamente

discriminativos, lo que valida que el feature selection por RFECV identificó las features más relevantes para la predicción.

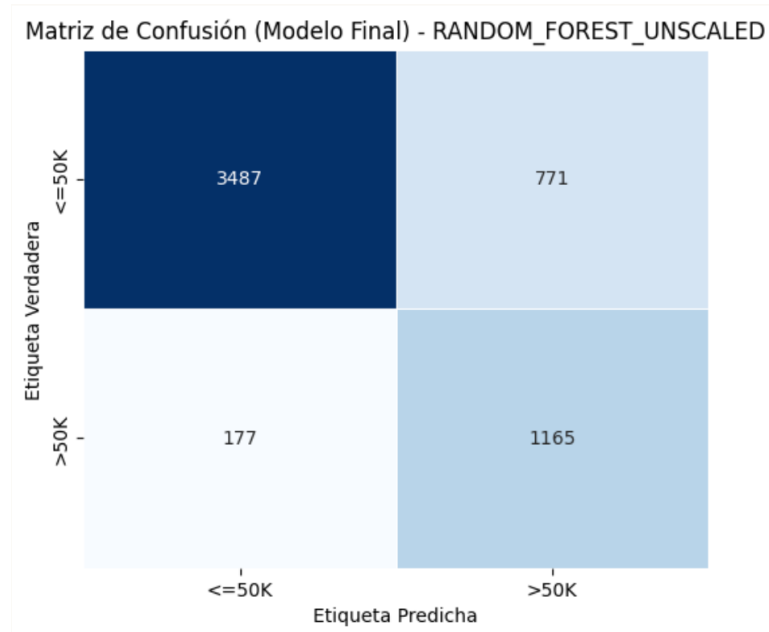


Figura 8: Matriz Confusión RF.

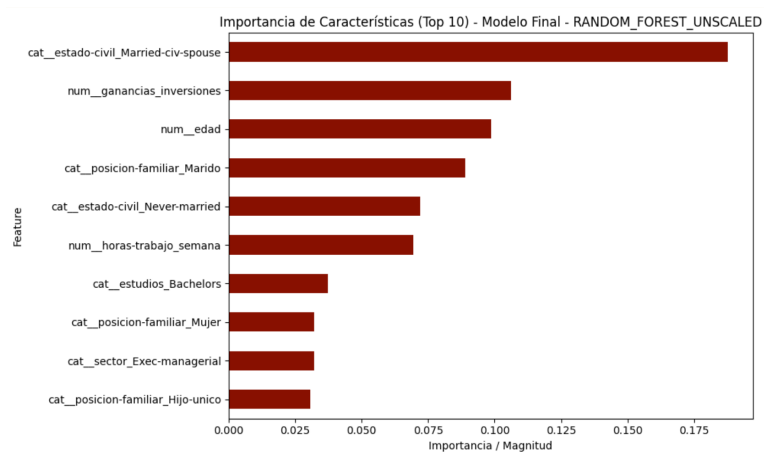


Figura 9: Importancia de Características RF.

4.1.3. Decisión 8.3: Aumento de Folds en Validación Cruzada (CV=5)

- Aunque en (`GridSearchCV` de múltiples modelos en `supervised.ipynb`) declaramos `CV=3` para reducir el alto coste computacional, al centrarnos exclusivamente en el modelo ganador (`Random Forest`) en la fase de implementación final (`best_model.ipynb`) **aumentamos la validación cruzada a CV=5**.
- Esto nos permite obtener una **evaluación de hiperparámetros más robusta y precisa** del modelo final, ya que la restricción del coste computacional por probar tantos modelos ya no la tenemos.

4.1.4. Decisión 8.4: Puntuación Final en la Competición

- Implementación de la Puntuación Final de Kaggle: El modelo `Random Forest Unscaled`, reentrenado con el conjunto completo de datos (`Train + Valid`), lo utilizamos para predecir las etiquetas en el set de `X_kaggle` y generar el archivo `test_labels.csv`.
- Rendimiento de la Competición: Al subir la (*submission*) de este archivo a la competición de Kaggle hemos obtenido una puntuación oficial (pública) de **0.832** (medida en F1-score), lo que valida el rendimiento y la capacidad de generalización del modelo óptimo frente a datos no vistos.

5. Herramientas y Recursos Utilizados

El desarrollo de este proyecto se ha llevado a cabo gracias a una serie de herramientas de software y recursos académicos para garantizar la calidad del trabajo, siguiendo la metodología de la asignatura.

- **Recursos de la Asignatura:** Las diapositivas y recursos de **Moodle** han sido la fuente metodológica principal para la aplicación de las técnicas de Aprendizaje Automático supervisado.
- **Entorno de Programación:** Hemos empleado el entorno de **Visual Studio Code** para la implementación y ejecución de los notebooks en Python, empleando bibliotecas estándar como Scikit-learn, Pandas, y Matplotlib.

- **Asistencia por IA:** Nos hemos apoyado en el uso de **Herramientas de Inteligencia Artificial** para generar código auxiliar (funciones de visualización de métricas como algunas gráficas realizadas en supervised.ipynb) y para resolver errores puntuales.
- **Documentación:** La memoria del proyecto la hemos creado utilizando **L^AT_EX** para cumplir con los estándares de documentación académica.